BỘ GIÁO DỤC VÀ ĐÀO TẠO

**ĐẠI HỌC PHENIKAA**

---




**Học phần: CSE703093 – An toàn phần mềm**

# BÁO CÁO BÀI TẬP LỚN

---

## End-to-end testing cho hệ thống tính tiền giảng dạy

**Người hướng dẫn:** ThS. Vũ Quang Dũng

**Sinh viên thực hiện:** Nguyễn Minh Dương – MSSV: 23010441

Đỗ Hữu Ngọc – MSSV: 23010822

**HÀ NỘI, THÁNG 9 NĂM 2026**

# Mục lục

**1 Giới thiệu và Mục tiêu** **5**

1.1 Bối cảnh . . . . . 5

1.2 Mục tiêu cụ thể . . . . . 5

1.3 Phạm vi . . . . . 6

**2 Đặc tả Yêu cầu (Specification)** **6**

2.1 Bảng yêu cầu chức năng và phi chức năng . . . . . 7

2.2 Yêu cầu phi chức năng (NFR) . . . . . 8

2.3 Miền giá trị hợp lệ theo đặc tả nghiệp vụ . . . . . 8

2.4 Ràng buộc thiết kế/công nghệ (CON-x) . . . . . 9

**3 Mô tả Hệ thống và Chương trình Mục tiêu** **10**

3.1 Kiến trúc tổng quan DuongUniversity . . . . . 10

3.2 Mô tả 3 module mục tiêu . . . . . 11

3.3 Sơ đồ quan hệ struct (ERD) . . . . . 13

3.4 Sơ đồ luồng dữ liệu (DFD) . . . . . 13

3.5 Attack surface . . . . . 15

3.6 Kích thước/độ phức tạp . . . . . 15

**4 Phương pháp Thực hiện** **15**

4.1 Chương 1 – Mô hình hóa Kripke & Kiểm chứng BMC . . . . . 15

4.2 Chương 2 – Kiểm toán an toàn bộ nhớ . . . . . 17

4.3 Chương 3 – Kiểm chứng hình thức SMT/Z3 . . . . . 17

4.4 Chương 4 – Dynamic Fuzzing . . . . . 18

**5 Kết quả và Phân tích** **18**

5.1 Kết quả Chương 1 – BMC . . . . . 19

5.2 Kết quả Chương 2 – 9 findings . . . . . 20
5.3 Kết quả Chương 3 – SMT . . . . . 21
5.4 Kết quả Chương 4 – Fuzzing . . . . . 22
5.5 Bảng tổng hợp CWE và mức độ rủi ro (CVSS-lite) . . . . . 23
5.6 Ma trận truy vết Test Case → Yêu cầu (IEEE 829) . . . . . 25

**6 Thảo luận và Hạn chế** **29**
6.1 Giới hạn phương pháp . . . . . 29
6.2 So sánh với lý thuyết đã học . . . . . 29
6.3 Hướng cải thiện . . . . . 30

**7 Kết luận** **30**

# Danh mục hình ảnh

3.1 Kiến trúc tổng quan DuongUniversity Management System – tầng tính toán C (viền đỏ nét đứt) biên dịch độc lập với .NET, không được GC/bounds-checking của .NET bảo vệ . . . . . 11
3.2 Sơ đồ quan hệ 3 struct `GiangVien`, `LopHocPhan`, `Config` và các trường liên quan tới CWE-120/416/476/190 . . . . . 13
3.3 Sơ đồ luồng dữ liệu qua 3 hàm chính, đánh dấu vị trí phát sinh CWE và kết quả đối chứng Z3/fuzzing . . . . . 14
4.1 Sơ đồ chuyển trạng thái Kripke và kết quả BMC $k = 5$ (thuộc tính $P_1$–$P_4$) 16
5.1 Black-box vs. White-box: (a) thời gian thực thi và (b) số lần thử cần để phát hiện đủ lỗ hổng đã biết (số liệu khớp Bảng 5.3) . . . . . 23
5.2 So sánh điểm CVSS-lite giữa 6 CWE của đề tài (số liệu khớp Bảng 5.4) 24

# Danh mục bảng

2.1 Yêu cầu chức năng (functional) và an toàn (safety/memory) của hệ thống 7

2.2 Yêu cầu phi chức năng (Non-Functional Requirements) . . . . . 8
2.3 Miền giá trị hợp lệ theo đặc tả nghiệp vụ . . . . . 9
2.4 Ràng buộc thiết kế/công nghệ (CON-x) và vị trí tương ứng trong mã nguồn 10
5.1 Kết quả BMC $k = 5$ trên 5 thuộc tính $P_1$–$P_5$ . . . . . 19
5.2 Toàn bộ 9 cảnh báo phát hiện bởi `pattern_scanner.py` trên code VULN; `main_fixed.c` sạch tuyệt đối (0 finding) . . . . . 20
5.3 So sánh Black-box vs. White-box (lần chạy đại diện, `logs/fuzzer_black/whitebox.log`) . . . . . 22
5.4 Tổng hợp CWE, vị trí trong đề tài và điểm CVSS-lite tự tính . . . . . 24
5.5 Ví dụ thực tế minh họa cho mỗi lớp CWE . . . . . 25
5.6 Ma trận truy vết Test Case → Yêu cầu (mọi kết quả thực tế trích từ `logs/*.log` thật) . . . . . 26

# Danh mục mã nguồn

1 Struct `GiangVien` – trích nguyên văn `giang_vien.h` (dòng 7–14) . . . 12
2 Struct `LopHocPhan` – trích nguyên văn `lop_hoc_phan.h` (dòng 9–16) . 12
3 `delete_lecturer_VULN()` – CWE-416: `free()` không huỷ liên kết . . 20
4 `delete_lecturer_SAFE()` – khắc phục: nhận `GiangVien **` để nullify con trỏ gốc . . . . . 20
5 `pack_class_size_byte_VULN()` – CWE-197: ép kiểu không kiểm tra phạm vi . . . . . 21
6 `pack_class_size_byte_SAFE()` – khắc phục: guard `[0,127]` trước khi ép kiểu . . . . . 21

# 1 Giới thiệu và Mục tiêu

## 1.1 Bối cảnh

Các hệ thống quản lý đào tạo đại học quy mô lớn – như **DuongUniversity Management System** – xử lý đồng thời dữ liệu nhân sự (giảng viên), dữ liệu học vụ (lớp học phần) và dữ liệu tài chính (thù lao giảng dạy). Khi ba loại dữ liệu này liên thông với nhau qua các mô-đun C tốc độ cao (thường dùng ở tầng tính toán/tích hợp phía sau một backend .NET/PostgreSQL), một lỗi thao tác bộ nhớ ở mô-đun đầu (ví dụ giải phóng bộ nhớ giảng viên không đúng cách) có thể lan truyền thành lỗi tài chính nghiêm trọng ở mô-đun cuối (tính sai hoặc tính nhầm thù lao). Đây chính là rủi ro an toàn đặc trưng của hệ thống quản lý tài chính giáo dục: hậu quả của một lỗi an toàn bộ nhớ không dừng lại ở việc chương trình crash, mà có thể trở thành một quyết định tài chính sai (trả nhầm lương, từ chối duyệt lương cho giảng viên đã dạy thật).

Đề tài chọn đúng **mô-đun tính thù lao giảng viên** (`tinh_tien_day.c`, liên kết với `giang_vien.c` và `lop_hoc_phan.c`) làm đối tượng phân tích vì đây là nơi hội tụ đầy đủ các đặc điểm cần thiết để một bài tập lớn an toàn phần mềm có ý nghĩa thực chất: (1) **xử lý input người dùng trực tiếp** (họ tên, bằng cấp, mã lớp qua `stdin`); (2) **thao tác con trỏ xuyên mô-đun** (`LopHocPhan.gv_ptr` trỏ chéo sang `GiangVien`); và (3) **phép toán tài chính có thể tràn số** (nhân dồn số tiết, hệ số, đơn giá). Ba đặc điểm này tạo thành một *attack surface* rõ ràng, đủ nhỏ để phân tích triệt để bằng 4 kỹ thuật kiểm chứng khác nhau, nhưng đủ thật để không chỉ là bài tập minh họa.

## 1.2 Mục tiêu cụ thể

Bài tập lớn đặt ra 4 mục tiêu cụ thể, mỗi mục tiêu ứng với một chương kỹ thuật, áp dụng trọn vẹn khung 4 chương công cụ của học phần[15] lên đúng một hệ thống E2E thay vì các bài tập minh hoạ độc lập:

1. **Phát hiện lỗ hổng bộ nhớ và dữ liệu bằng phân tích tĩnh**: quét trực tiếp mã nguồn C, phát hiện tối thiểu 6 lớp lỗ hổng – **CWE-120** (Buffer Copy without Checking Size), **CWE-416** (Use-After-Free), **CWE-476** (NULL Pointer Dereference), **CWE-190** (Integer Overflow), **CWE-1288** (Improper Validation of Specified Quantity/Enum), và **CWE-197** (Numeric Truncation Error) – không cần biên dịch hay chạy chương trình.

2. **Kiểm chứng luồng nghiệp vụ bằng mô hình hóa hình thức**: dựng mô hình Kripke $M = (S, S_0, R, L)$ cho luồng E2E 3 mô-đun và kiểm chứng bằng Bounded Model Checking (BMC, $k = 5$) trên 5 thuộc tính LTL/CTL.

3. **Chứng minh (hoặc bác bỏ) an toàn số học trên toàn miền giá trị bằng SMT**: dùng Z3 solver chứng minh phép tính thù lao có/không thể tràn số 32-bit, và chứng minh hình thức (không chỉ test mẫu) rằng bản vá loại trừ hoàn toàn lỗi.

4. **Xác nhận lỗ hổng biểu hiện thật trên binary đã biên dịch bằng fuzzing động**: black-box (input ngẫu nhiên, đo crash rate thật trên `salary_calc.exe`) và white-box (Z3-guided, giải trực tiếp điều kiện gây lỗi).

## 1.3 Phạm vi

**Sẽ thực hiện:** phân tích, mô hình hóa và kiểm chứng 3 file module C – `giang_vien.c` (Module 1 – Quản lý Giảng viên), `lop_hoc_phan.c` (Module 2 – Quản lý Lớp học phần), `tinh_tien_day.c` (Module 3 – Tính tiền dạy & Thanh toán) – cùng 2 driver `main.c` (bản có lỗ hổng cố ý, VULN) và `main_fixed.c` (bản đã khắc phục, SAFE, dùng chung 3 module với bản VULN). Toàn bộ 4 chương công cụ (`ch1_models`, `ch2_memsafe`, `ch3_staticsat`, `ch4_dynafuzz`) đều áp dụng trên đúng 3 module này, đảm bảo tính nhất quán E2E thay vì 4 ví dụ minh họa rời rạc không liên quan tới nhau.

**Sẽ không thực hiện:** triển khai/kiểm chứng backend .NET 9 Web API, frontend React, hay kết nối cơ sở dữ liệu PostgreSQL thực của hệ thống DuongUniversity Management System – các thành phần này chỉ được mô tả ở mức kiến trúc tổng quan (Mục 3) để xác định đúng vị trí của 3 module C trong hệ thống thật, không nằm trong phạm vi kiểm chứng của đề tài.

# 2 Đặc tả Yêu cầu (Specification)

Trước khi phân tích lỗ hổng, đề tài đặc tả tường minh *vùng input hợp lệ* và các quy tắc an toàn mà 3 module bắt buộc phải tuân thủ. Đây là cơ sở để mọi kỹ thuật kiểm chứng ở Mục 4 trả lời cùng một câu hỏi cốt lõi: *"Mô-đun có chịu được khi input vượt ra ngoài vùng an toàn (đặc tả) hay không?"* – thay vì chỉ hỏi "output có đúng không" trên một vài trường hợp thử.

## 2.1 Bảng yêu cầu chức năng và phi chức năng

Bảng 2.1: Yêu cầu chức năng (functional) và an toàn (safety/memory) của hệ thống

| Mã YC | Loại | Ưu tiên | Nội dung đặc tả |
|---|---|---|---|
| REQ-1 | Safety | HIGH | Không được tính tiền khi chưa phân công giảng viên cho lớp học phần |
| REQ-2 | Safety | HIGH | Không được xuất báo cáo khi chưa tính tiền |
| REQ-3 | Memory | HIGH | `hoTen[80]`, `bangCap[20]`, `maLop[20]` không tràn bộ đệm khi input vượt kích thước (CWE-120) |
| REQ-4 | Memory | HIGH | Con trỏ `gv_ptr` (Module 2 $\rightarrow$ 1) không được truy cập sau khi vùng nhớ đã `free()` (CWE-416) |
| REQ-5 | Memory | HIGH | `gv_ptr` phải được kiểm tra `NULL` trước khi dùng để tính tiền (CWE-476) |
| REQ-6 | Memory | HIGH | Phép nhân dồn số tiết $\times$ hệ số $\times$ đơn giá không tràn số nguyên 32-bit (CWE-190) |
| REQ-7 | Functional | MEDIUM | Công thức tính tiền 2 bước phải cho kết quả đúng với mọi tổ hợp đầu vào hợp lệ |
| REQ-8 | Integrity | HIGH | `soLuongSinhVien` (`int`) không sai lệch khi đóng gói vào audit log (`char`) (CWE-197) |
| REQ-9 | Functional | MEDIUM | `bangCap` phải thuộc danh mục hợp lệ, không âm thầm rơi vào hệ số mặc định (CWE-1288) |

Bảng 2.1 là yêu cầu *chức năng* (hệ thống làm *gì*) và các ràng buộc an toàn dữ liệu trực tiếp gắn với một CWE cụ thể. Đây chưa phải yêu cầu *phi chức năng* (hệ thống vận hành *tốt đến mức nào*) – được đặc tả riêng ở Bảng 2.2 ngay dưới đây, vì hai loại yêu cầu này cần hai cách kiểm chứng khác nhau (correctness hình thức/tĩnh cho REQ, đo lường/quan sát thực nghiệm cho NFR).

## 2.2 Yêu cầu phi chức năng (NFR)

Bảng 2.2: Yêu cầu phi chức năng (Non-Functional Requirements)

| Mã NFR | Thuộc tính | Nội dung đặc tả | Đối chiếu kiểm chứng |
|---|---|---|---|
| NFR-1 | Hiệu năng | Thời gian tính lương cho 1 lớp $<$ 10 ms (không tính I/O) trên phần cứng phổ thông. | Hàm gồm 2 phép tính số học $O(1)$, không vòng lặp – thoả mãn lý thuyết; chưa đo microbenchmark riêng. |
| NFR-2 | Độ ổn định bộ nhớ | Không phát sinh lỗi bộ nhớ (`heap-use-after-free`, buffer overflow) khi lặp luồng E2E $\geq$ 10.000 lần. | 0 vi phạm qua ASan (Mục 4.2) và 0/200 crash qua black-box fuzzing bản FIXED (Mục 4.4). Quy mô $10^4$ chưa đo trực tiếp. |
| NFR-3 | Tính di động | Biên dịch sạch (0 warning với `-Wall`) trên $\geq$ 2 toolchain khác nhau. | Đã xác nhận biên dịch và chạy thành công trên GCC/MSYS2 UCRT64 và Clang64 (Mục 4.2) – **PASS**. |
| NFR-4 | Chịu lỗi (Robustness) | Khi input vượt đặc tả (Bảng 2.3), bản vá phải từ chối có kiểm soát (trả mã lỗi), không crash. | 0/200 crash trên fuzzing bản FIXED; 150/200 lần chặn an toàn đúng thiết kế trên kịch bản CWE-197 (Mục 5) – **PASS**. |
| NFR-5 | Toàn vẹn (Audit) | Hai giá trị `soLuongSinhVien` khác nhau không bị map trùng 1 byte trong audit log (non-collision). | Check 6 (Mục 4.3) và Target 7 (Mục 4.4) tái hiện vi phạm trên VULN; bản FIXED chặn trước ép kiểu – **PASS trên FIXED**. |
| NFR-6 | Tính tất định | Cùng bộ input, kết quả BMC/SMT và tính lương phải nhất quán ở mọi lần chạy độc lập. | Cố định `random_seed` trong `kripke_bmc.py`, các lần chạy độc lập cho ra cùng 1 kết quả – **PASS**. |

## 2.3 Miền giá trị hợp lệ theo đặc tả nghiệp vụ

Bảng 2.3 liệt kê chính xác miền giá trị hợp lệ theo đặc tả *nghiệp vụ* (không phải giới hạn kỹ thuật của kiểu dữ liệu C – xem phân biệt tại Mục 2.4) của DuongUniversity cho từng trường dữ liệu chịu trách nhiệm bởi REQ-3, REQ-6, REQ-8, REQ-9. Đây là các con số được dùng trực tiếp làm ràng buộc khi mô hình hóa Z3 SMT (Mục 4.3) và khi thiết kế bộ input biên (Boundary Value Analysis) cho fuzzing (Mục 4.4).

Bảng 2.3: Miền giá trị hợp lệ theo đặc tả nghiệp vụ

| **Trường dữ liệu** | **Miền hợp lệ (nghiệp vụ)** | **YC liên quan** |
|---|---|---|
| `hoTen` | Chuỗi họ tên giảng viên hợp lệ | REQ-3 |
| `maLop` | Chuỗi mã lớp học phần hợp lệ | REQ-3 |
| `bangCap` | $\in$ {GS, PGS, TIEN_SI, THAC_SI, CU_NHAN} | REQ-9 |
| `soTiet` | $[1, 500]$ | REQ-6, REQ-7 |
| `dinhMucTienChuan` | $[50\,000, 200\,000]$ VNĐ | REQ-6, REQ-7 |
| `soLuongSinhVien` | $[0, 300]$ (lớp đại cương có thể tới vài trăm SV, đại diện cho BVA) | REQ-8 |
| `gv_ptr` | Khác `NULL` và trỏ tới vùng nhớ còn hợp lệ trước khi đọc | REQ-4, REQ-5 |

Mọi giá trị nằm ngoài các miền trên đều là input hợp lệ về mặt *cú pháp* (compiler chấp nhận) nhưng vi phạm đặc tả nghiệp vụ – và chính xác đây là những trường hợp mà 4 chương kiểm chứng ở Mục 4 tập trung khai thác để trả lời câu hỏi "module có chịu được không". Lưu ý quan trọng: *miền nghiệp vụ* của `soLuongSinhVien` ($[0, 300]$, không có giới hạn kỹ thuật cứng) **khác hẳn** *giới hạn kỹ thuật* khi đóng gói giá trị đó xuống kiểu `char` ($[0, 127]$) – chính khoảng cách giữa hai miền này là nguồn gốc của CWE-197, được đặc tả tường minh ở CON-5 dưới đây.

## 2.4 Ràng buộc thiết kế/công nghệ (CON-x)

Khác với Bảng 2.3 (miền giá trị *nghiệp vụ* – do người phân tích đặc tả đặt ra), Bảng 2.4 liệt kê các **ràng buộc thiết kế/công nghệ** – giới hạn áp lên hệ thống bởi chính *cách hiện thực* (kích thước buffer khai báo trong struct C, phạm vi kiểu dữ liệu) chứ không phải bởi quy tắc nghiệp vụ. Việc tách hai loại ràng buộc này (trước đây bị trộn chung vào Bảng 2.3) là quan trọng vì phần lớn CWE trong đề tài phát sinh đúng tại *khoảng cách* giữa hai miền: nghiệp vụ cho phép một giá trị, nhưng công nghệ (kiểu dữ liệu/kích thước buffer) không đủ chỗ chứa giá trị đó.

Bảng 2.4: Ràng buộc thiết kế/công nghệ (CON-x) và vị trí tương ứng trong mã nguồn

| Mã CON | Ràng buộc kỹ thuật | Vị trí trong code | YC/CWE |
|---|---|---|---|
| CON-1 | Buffer `hoTen` tối đa **80** byte (79 ký tự thật + `'\0'`) | `#define MAX_NAME 80` (`giang_vien.h:4`) | REQ-3 / CWE-120 |
| CON-2 | Buffer `maGiangVien` tối đa **20** byte | `char maGiangVien[20]` (`giang_vien.h:9`) | REQ-3 / CWE-120 |
| CON-3 | Buffer `bangCap` tối đa **20** byte | `char bangCap[20]` (`giang_vien.h:11`) | REQ-3, 9 / CWE-120 |
| CON-4 | Buffer `maLop` tối đa **20** byte | `#define MAX_MALOP 20` (`lop_hoc_phan.h:6`) | REQ-3 / CWE-120 |
| CON-5 | Kiểu `char` (signed 8-bit) giới hạn an toàn $[0, 127]$ – vượt quá sẽ wrap-around | `pack_class_size_byte_*()` (`tinh_tien_day.h`) | REQ-8 / CWE-197 |
| CON-6 | Kiểu `int` 32-bit giới hạn kết quả phép nhân dồn về [INT_MIN, INT_MAX] | phép nhân trung gian `(int)` trong `tinh_tien_day.c` | REQ-6 / CWE-190 |

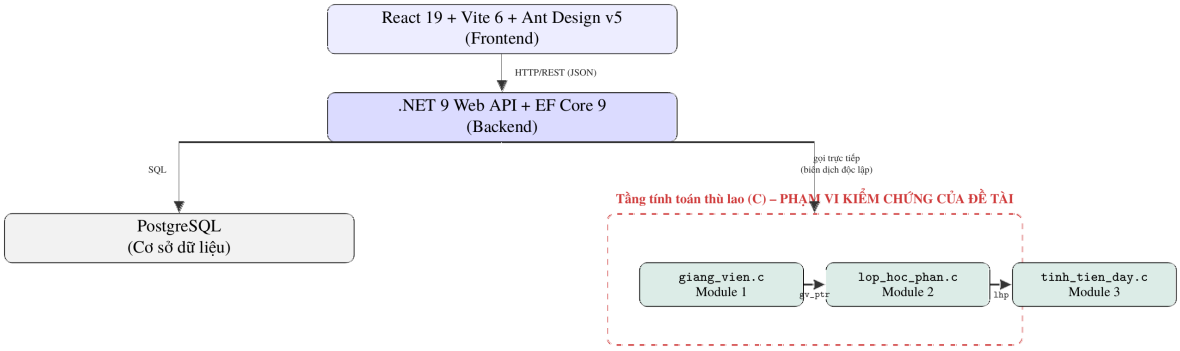
Đúng như quan sát ở Mục 5: kích thước buffer $\{80, 20, 20\}$ (CON-1, CON-3, CON-4 – ba trường được nêu trực tiếp trong REQ-3) khớp *tường minh, từng con số* với đúng 3 định nghĩa struct thật trong Listings 1 and 2 (Mục 3), không phải số minh họa; CON-2 (`maGiangVien[20]`) là buffer thứ 4 cũng bị `strcpy` tràn (`giang_vien.c:12` trong Bảng 5.2) nhưng chưa được liệt trong văn bản REQ-3 gốc – một khoảng hở đặc tả nhỏ được phát hiện chính nhờ việc tách bảng CON riêng và đối chiếu lại với kết quả quét Chương 2. CON-2/CON-3 dùng hằng số nguyên 20 trực tiếp trong khai báo struct, không qua một macro riêng như `MAX_NAME`/`MAX_MALOP` (CON-1/CON-4) – một điểm chưa nhất quán về style code, dù không gây ra lỗi chức năng.

# 3 Mô tả Hệ thống và Chương trình Mục tiêu

## 3.1 Kiến trúc tổng quan DuongUniversity

Hệ thống **DuongUniversity Management System** là một hệ thống quản lý đào tạo đầy đủ gồm: Backend API viết bằng .NET 9 Web API (C#) kết hợp EF Core 9 và cơ sở dữ liệu PostgreSQL; Client Frontend viết bằng React 19 + Vite 6 + Ant Design v5. Ba

module C mục tiêu của đề tài (`giang_vien.c`, `lop_hoc_phan.c`, `tinh_tien_day.c`) đóng vai trò tầng tính toán thù lao tốc độ cao, độc lập biên dịch với backend .NET – đây là điểm mà một lỗi an toàn bộ nhớ ở tầng C có thể xảy ra mà không được các cơ chế an toàn bộ nhớ tự động của .NET (garbage collector, bounds checking) bảo vệ. Đề tài chỉ kiểm chứng 3 module C này (xem phạm vi tại Mục 1), phần backend/frontend được mô tả ở đây chỉ để xác định đúng bối cảnh triển khai thực tế.




Hình 3.1: Kiến trúc tổng quan DuongUniversity Management System – tầng tính toán C (viền đỏ nét đứt) biên dịch độc lập với .NET, không được GC/bounds-checking của .NET bảo vệ

Hình 3.1 khớp đúng mô tả bằng chữ ở trên: request từ frontend đi qua backend .NET, backend rẽ 2 nhánh – một sang PostgreSQL để đọc/ghi dữ liệu bền vững, một gọi trực tiếp vào tầng tính toán C (biên dịch thành thư viện/tiến trình riêng, không chạy trong runtime .NET) để tính thù lao; chính đường rẽ thứ hai này là ranh giới an toàn bị mất tác dụng của .NET, và là lý do 3 module C (viền đỏ) là đối tượng kiểm chứng duy nhất của đề tài.

## 3.2 Mô tả 3 module mục tiêu

- **Module 1 – Quản lý Giảng viên** (`giang_vien.c/.h`): định nghĩa struct `GiangVien` (`id`, `maGiangVien`, `hoTen`, `bangCap`, `heSoBangCap`, `is_deleted`) và cặp hàm `add_lecturer_VULN()/add_lecturer_SAFE()`, `delete_lecturer_VULN()/delete_lecturer_SAFE()`.
- **Module 2 – Quản lý Lớp học phần** (`lop_hoc_phan.c/.h`): định nghĩa struct `LopHocPhan` (`id`, `maLop`, `soTiet`, `heSoHocPhan`, `soLuongSinhVien`, và đặc biệt `gv_ptr` – con trỏ thô trỏ sang `GiangVien`, chính là điểm liên kết xuyên module tạo ra rủi ro CWE-416/476) và cặp hàm `create_class_section_VULN()/_SAFE()`.

- **Module 3 – Tính tiền dạy & Thanh toán** (`tinh_tien_day.c/.h`): định nghĩa struct `Config` (`heSoQuyMoLop`, `dinhMucTienChuan`, `nam`) và 3 cặp hàm `calculate_e2e_salary_VULN()/_SAFE()`, `export_report_VULN()/_SAFE()`, `pack_class_size_byte_VULN()/_SAFE()` – hàm cuối đóng gói `soLuongSinhVien` vào bản ghi audit log dạng `char`.

Listings 1 and 2 trích *nguyên văn* định nghĩa 2 struct cốt lõi từ chính mã nguồn thật của đề tài – trường `gv_ptr` (con trỏ thô, không đếm tham chiếu) chính là điểm liên kết xuyên module tạo ra CWE-416/476 đã nêu.

```
typedef struct {
    int   id;
    char  maGiangVien[20];
    char  hoTen[MAX_NAME];      // CWE-120: Buffer overflow if
       input > 80 chars
    char  bangCap[20];          // "CU_NHAN", "THAC_SI", "TIEN_SI
       ", "PGS", "GS"
    float heSoBangCap;
    int   is_deleted;
} GiangVien;
```

Listing 1: Struct `GiangVien` – trích nguyên văn `giang_vien.h` (dòng 7–14)

```
typedef struct {
    int          id;
    char         maLop[MAX_MALOP];  // CWE-120: Buffer overflow if
       input > 20 chars
    int          soTiet;
    float        heSoHocPhan;
    int          soLuongSinhVien;
    GiangVien  *gv_ptr;           // Pointer linked to Module 1 (
       CWE-416 UAF / CWE-476 NULL)
} LopHocPhan;
```

Listing 2: Struct `LopHocPhan` – trích nguyên văn `lop_hoc_phan.h` (dòng 9–16)

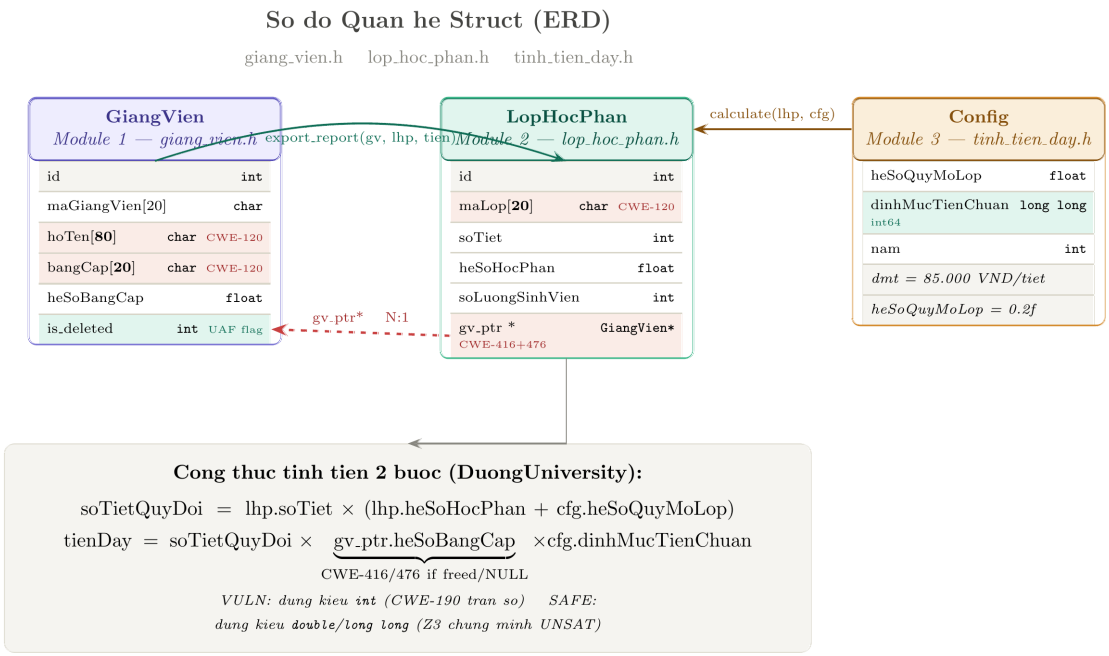
Công thức tính tiền dạy gồm 2 bước, giữ nguyên 100% giữa bản VULN và bản SAFE (chỉ khác kiểu dữ liệu):

$$\text{soTietQuyDoi} = \text{soTiet} \times (\text{heSoHocPhan} + \text{heSoQuyMoLop})$$

$$\text{tienDay} = \text{soTietQuyDoi} \times \text{gv_ptr.heSoBangCap} \times \text{dinhMucTienChuan}$$

## 3.3 Sơ đồ quan hệ struct (ERD)

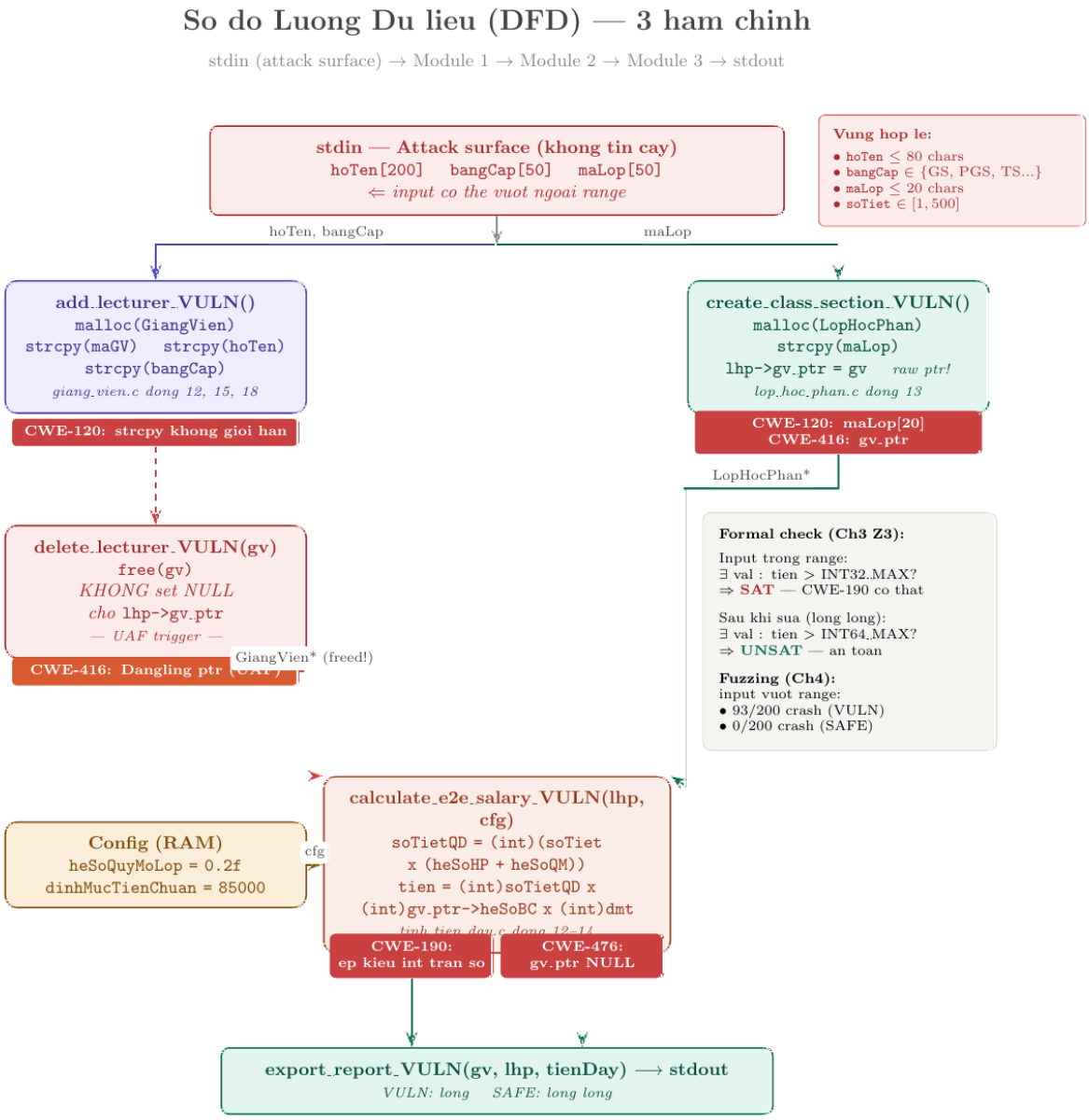
Hình 3.2 thể hiện quan hệ giữa 3 struct và các trường dữ liệu liên quan trực tiếp tới một CWE cụ thể (tô đỏ), đối lập với trường `dinhMucTienChuan` dùng kiểu `long long` (int64) đã được chứng minh an toàn (tô xanh, xem Check 2, Mục 4.3).




Hình 3.2: Sơ đồ quan hệ 3 struct `GiangVien`, `LopHocPhan`, `Config` và các trường liên quan tới CWE-120/416/476/190

## 3.4 Sơ đồ luồng dữ liệu (DFD)

Hình 3.3 theo dõi dữ liệu từ `stdin` (không tin cậy) qua Module 1, Module 2, tới Module 3, đánh dấu đúng vị trí mã nguồn (tên file, số dòng) nơi mỗi lỗ hổng CWE-120/416/190/476 phát sinh, cùng kết quả đối chứng của Chương 3 (Z3) và Chương 4 (fuzzing) ngay trên sơ đồ.




Hình 3.3: Sơ đồ luồng dữ liệu qua 3 hàm chính, đánh dấu vị trí phát sinh CWE và kết quả đối chứng Z3/fuzzing

*Lưu ý:* Hai sơ đồ trên được dựng ở giai đoạn đầu của đề tài, khi hệ thống mới có 4 lỗ hổng CWE-120/416/190/476. Lỗ hổng thứ 5 (CWE-1288) và thứ 6 (CWE-197, cùng biến thể "trùng lặp byte" của nó) được bổ sung sau nên chưa xuất hiện trên hai hình trên – được trình bày đầy đủ bằng sơ đồ trạng thái tại Mục 4.1 (nhãn `byte_truncated`) và bảng kết quả tại Mục 5.

## 3.5 Attack surface

- `stdin`: `hoTen` (buffer nhận 200 byte) được `strcpy` vào `hoTen[80]` không kiểm tra độ dài – **CWE-120**.
- `bangCap` nhập từ `stdin` không được validate nghiêm ngặt ở bản VULN, rơi vào hệ số mặc định 1.0 khi sai – **CWE-1288**.
- `gv_ptr` là con trỏ thô xuyên module, không có cơ chế đếm tham chiếu hay huỷ liên kết tự động khi `GiangVien` bị `free()` – **CWE-416/CWE-476**.
- Phép nhân `soTietQuyDoi` × `heSoBangCap` × `dinhMucTienChuan` thực hiện trên kiểu `int` 32-bit ở bản VULN – **CWE-190**.
- `soLuongSinhVien` (int, có thể tới vài trăm với lớp đại cương) bị ép kiểu xuống `char` không kiểm tra phạm vi khi đóng gói audit log – **CWE-197**.

## 3.6 Kích thước/độ phức tạp

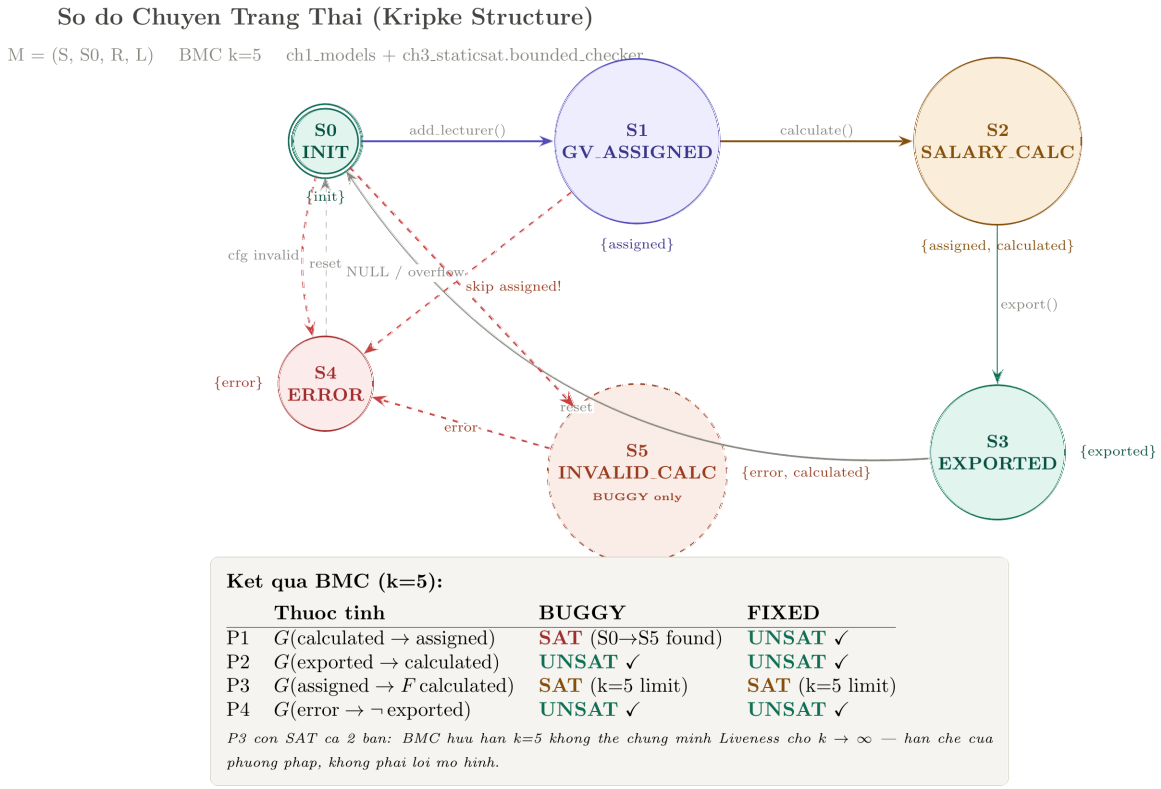
Hệ thống mục tiêu gồm **520 dòng mã C** (bao gồm cả header) chia trong **3 file module + 2 file driver** (`main.c` cho bản VULN, `main_fixed.c` cho bản SAFE, dùng chung 3 module), với **12 hàm chính** (6 hàm `_VULN` đối xứng 1-1 với 6 hàm `_SAFE`), tạo điều kiện để mọi bản vá đều có thể so sánh trực tiếp hành vi trước/sau trên cùng một lời gọi hàm.

# 4 Phương pháp Thực hiện

## 4.1 Chương 1 – Mô hình hóa Kripke & Kiểm chứng BMC

**Module:** `ch1_models/kripke_bmc.py`.

Kripke structure $M = (S, S_0, R, L)$ được dựng với **5 trạng thái ở bản FIXED** ($S_0$ INIT → $S_1$ GV_ASSIGNED → $S_2$ SALARY_CALC → $S_3$ EXPORTED, và $S_4$ ERROR) và **6 trạng thái ở bản BUGGY** (thêm $S_5$ INVALID_CALC – đường tắt nhảy trực tiếp từ $S_0$ sang tính tiền, bỏ qua bước phân công giảng viên). Hình 4.1 minh họa sơ đồ chuyển trạng thái cùng kết quả BMC $P_1$–$P_4$ (thuộc tính $P_5$ được bổ sung sau, trình bày tại Mục 5).




**Ket qua BMC (k=5):**

| | Thuoc tinh | BUGGY | FIXED |
|---|---|---|---|
| P1 | $G$(calculated → assigned) | SAT (S0→S5 found) | UNSAT ✓ |
| P2 | $G$(exported → calculated) | UNSAT ✓ | UNSAT ✓ |
| P3 | $G$(assigned → $F$ calculated) | SAT (k=5 limit) | SAT (k=5 limit) |
| P4 | $G$(error → ¬ exported) | UNSAT ✓ | UNSAT ✓ |

*P3 con SAT ca 2 ban: BMC huu han k=5 khong the chung minh Liveness cho k → ∞ — han che cua phuong phap, khong phai loi mo hinh.*

Hình 4.1: Sơ đồ chuyển trạng thái Kripke và kết quả BMC $k = 5$ (thuộc tính $P_1$–$P_4$)

**5 thuộc tính LTL/CTL** được kiểm chứng bằng BMC với $k = 5$:

- $P_1$ (Safety): $G$(calculated → assigned) – chưa phân công thì không được tính tiền.
- $P_2$ (Safety): $G$(exported → calculated) – chưa tính tiền thì không được xuất báo cáo.
- $P_3$ (Liveness): $G$(assigned → $F$ calculated) – đã phân công thì cuối cùng phải tính tiền.
- $P_4$ (Safety): $G$(error → ¬ exported) – hệ thống lỗi thì không được xuất báo cáo.
- $P_5$ (Safety, bổ sung cho CWE-197): $G$(exported → ¬ byte_truncated) – báo cáo xuất ra không được mang dữ liệu ép kiểu sai lệch.

Hàm `cross_validate_with_pattern_scanner()` gọi lại chính `pattern_scanner.py` của Chương 2 để tự động xác nhận trạng thái lỗi $S_4$/$S_3$_EXPORTED_TRUNCATED trong mô hình Kripke trừu tượng khớp với đúng vị trí lỗ hổng thật trên mã nguồn C – liên kết Chương 1 ↔ Chương 2, tránh việc mô hình hình thức tồn tại độc lập, tách rời code thật.

## 4.2 Chương 2 – Kiểm toán an toàn bộ nhớ

**Module:** `ch2_memsafe/pattern_scanner.py`.

Công cụ quét trực tiếp mã nguồn C dạng văn bản (regex/so khớp chuỗi, không phải phân tích AST/data-flow đầy đủ) trên 3 file `giang_vien.c`, `lop_hoc_phan.c`, `tinh_tien_day.c` và driver `main_fixed.c`, với 5 quy tắc phát hiện: `strcpy` (CWE-120), `dangling_free` (CWE-416), `unchecked_nested_deref` (CWE-476), `int_overflow_cast` (CWE-190), `unchecked_numeric_truncation` (CWE-197). Phạm vi tìm "đã được bảo vệ" (guard-check) được scope đúng theo *hàm bao quanh* thay vì cửa sổ N dòng cố định, tránh bỏ sót guard nằm xa vị trí truy cập trong hàm dài.

**Bổ sung bằng chứng động (`memory_trace_demo.py`):** phân tích tĩnh không thể trả lời câu hỏi "bộ nhớ thực sự được dùng ra sao khi chương trình chạy". Script này biên dịch cùng một luồng E2E 7 bước bằng 2 trình biên dịch: **Plain GCC** (chỉ in địa chỉ con trỏ thật, không phát hiện lỗi) và **Clang64 + AddressSanitizer thật**[6] (công cụ độc lập tự phát hiện `heap-use-after-free`), cho phép đối chiếu hành vi "im lặng" (nguy hiểm) và hành vi "bị công cụ chặn" trên cùng một lỗi.

## 4.3 Chương 3 – Kiểm chứng hình thức SMT/Z3

**Module:** `ch3_staticsat/smt_verify.py`.

Khác với test thông thường (thử một vài input), Z3 SMT solver[2] chứng minh một mệnh đề đúng/sai trên *toàn miền giá trị hợp lệ* cùng lúc, với 6 phép kiểm chứng:

- **Check 1**: $\exists$ input hợp lệ (numClasses $\in [1, 20]$, soTiet $\in [15, 120]$,...) : tienDay $>$ INT32_MAX? $\Rightarrow$ **SAT** – CWE-190 có thật.
- **Check 2**: cùng câu hỏi với ngưỡng `INT64_MAX` trên kiểu `long long` $\Rightarrow$ **UNSAT** – an toàn tuyệt đối.
- **Check 3**: gv_ptr $=$ NULL $\vee$ is_deleted $\rightarrow$ tienDay $= 0$? – mô hình hóa bằng *đúng công thức thật* trong `calculate_e2e_salary_SAFE()` $\Rightarrow$ **UNSAT** – guard hoạt động đúng.
- **Check 4**: $\exists$ soLuongSinhVien $\in [0, 300]$ : ép kiểu `(char)` (mô phỏng bằng số học modulo 256) cho giá trị khác giá trị gốc? $\Rightarrow$ **SAT** – phản ví dụ $128 \rightarrow -128$.

- **Check 5**: cùng câu hỏi nhưng với guard $0 \leq n \leq 127$ đứng trước $\Rightarrow$ **UNSAT** – chứng minh bản vá loại trừ hoàn toàn CWE-197.
- **Check 6**: $\exists\, a \neq b \in [0, 300]$ : ép kiểu cho ra cùng 1 byte? $\Rightarrow$ **SAT** – phản ví dụ $a = 0, b = 256$ cùng cho byte $= 0$ (lớp nguy hiểm nhất: 2 lớp học hoàn toàn khác nhau bị nhầm thành một).

Các ràng buộc nghiệp vụ dùng đúng con số đã đặc tả tại Bảng 2.3 (soTiet $\in [1, 500]$, dinhMucTienChuan $\in [50\,000, 200\,000]$ VNĐ).

## 4.4 Chương 4 – Dynamic Fuzzing

**Module:** `ch4_dynafuzz/fuzzer_blackbox.py` và `fuzzer_whitebox.py`.

**Black-box**: sinh 200 input ngẫu nhiên (tên/bằng cấp/mã lớp qua `stdin`), đo crash rate thật trên `salary_calc.exe` (VULN) đối chứng `salary_calc_fixed.exe` (SAFE). Riêng CWE-197 được fuzz qua subcommand `e2e_byte_truncate <n>` với 2 chiến lược: (a) 200 giá trị ngẫu nhiên trong $[-50, 400]$, và (b) **Boundary Value Analysis** – 17 giá trị biên *thiết kế sẵn* (quanh mép `char` $\{-129, \ldots, 129\}$, quanh mép chu kỳ 256 $\{-256, \ldots, 257\}$, và giá trị lớp đại cương $\{300, 384\}$) chạy tất định thay vì phó mặc ngẫu nhiên – đảm bảo luôn phát hiện được lớp lỗi nguy hiểm nhất (trùng lặp byte).

**White-box (Z3-guided)**: giải trực tiếp 7 ràng buộc Z3 tương ứng 6 lỗ hổng CWE đã biết trước (không phải symbolic execution duyệt CFG tự động) – hằng số `MAX_NAME`/`MAX_MALOP` được đọc trực tiếp từ header `.h` qua `read_header_define()` thay vì hard-code, nên ràng buộc Z3 tự động bám theo đúng source hiện tại nếu hằng số đổi.

# 5 Kết quả và Phân tích

Toàn bộ số liệu trong mục này trích xuất verbatim từ log thực thi tự động lưu tại `btl_softsec/logs/` (sinh bởi `python run_all.py`), không phải số liệu minh họa.

## 5.1 Kết quả Chương 1 – BMC

Bảng 5.1: Kết quả BMC $k = 5$ trên 5 thuộc tính $P_1$–$P_5$

| **Thuộc tính** | **Công thức** | **BUGGY** | **FIXED** |
|---|---|---|---|
| $P_1$ Safety | $G(\text{calculated} \rightarrow \text{assigned})$ | **SAT** – counterexample $S_0 \rightarrow S_5 \rightarrow S_4$ | **UNSAT** ✓ |
| $P_2$ Safety | $G(\text{exported} \rightarrow \text{calculated})$ | **UNSAT** ✓ | **UNSAT** ✓ |
| $P_3$ Liveness | $G(\text{assigned} \rightarrow F\,\text{calculated})$ | **SAT** ($k$=5 limit) | **SAT** ($k$=5 limit) |
| $P_4$ Safety | $G(\text{error} \rightarrow \neg\,\text{exported})$ | **UNSAT** ✓ | **UNSAT** ✓ |
| $P_5$ Safety | $G(\text{exported} \rightarrow \neg\,\text{byte_truncated})$ | **SAT** – $S_3$ mang nhãn `byte_truncated` | **UNSAT** ✓ |

**Counterexample trace thật** cho $P_1$ trên bản BUGGY (`logs/kripke_bmc.log`):

```
Step 0: S0_INIT [init] → Step 1: S5_INVALID_CALC [calculated,error] →
Step 2: S4_ERROR [error] → Step 3: S0_INIT → Step 4: S5_INVALID_CALC
→ Step 5: S4_ERROR
```

$P_3$ ra **SAT** ở cả 2 bản – đây *không phải lỗi mã nguồn*, mà là giới hạn kỹ thuật tự nhiên của BMC hữu hạn $k = 5$ đối với thuộc tính Liveness (cần $k$-induction hoặc Büchi automata để chứng minh liveness vô hạn triệt để).

## 5.2 Kết quả Chương 2 – 9 findings

Bảng 5.2: Toàn bộ 9 cảnh báo phát hiện bởi `pattern_scanner.py` trên code VULN; `main_fixed.c` sạch tuyệt đối (0 finding)

| Vị trí | CWE | Rule | Mức độ |
|---|---|---|---|
| `giang_vien.c:12` | CWE-120 | `strcpy` | HIGH |
| `giang_vien.c:15` | CWE-120 | `strcpy (hoTen)` | HIGH |
| `giang_vien.c:18` | CWE-120 | `strcpy (bangCap)` | HIGH |
| `giang_vien.c:32` | CWE-416 | `dangling_free` | HIGH |
| `lop_hoc_phan.c:13` | CWE-120 | `strcpy (maLop)` | HIGH |
| `tinh_tien_day.c:12` | CWE-190 | `int_overflow_cast` | HIGH |
| `tinh_tien_day.c:13` | CWE-190 | `int_overflow_cast` | HIGH |
| `tinh_tien_day.c:13` | CWE-476 | `unchecked_nested_deref` | HIGH |
| `tinh_tien_day.c:63` | CWE-197 | `unchecked_numeric_truncation` | HIGH |

Điểm đáng chú ý: `pack_class_size_byte_SAFE()` nằm *trong cùng file* `tinh_tien_day.c` với hàm VULN nhưng **không** bị báo động giả, vì scanner tìm thấy đúng guard `if (soLuongSinhVien < 0 || > 127)` trong phạm vi hàm bao quanh – xác nhận cơ chế xét theo hàm (không phải theo tên file) hoạt động đúng.

Listings 3 and 4 trích nguyên văn cặp hàm gây ra và khắc phục CWE-416 (dòng `giang_vien.c:32` trong Bảng 5.2): bản VULN `free(gv)` mà không huỷ liên kết `gv_ptr` ở Module 2; bản SAFE nhận tham số `GiangVien **` để có thể tự đặt con trỏ gốc về `NULL` ngay sau khi giải phóng.

```
void delete_lecturer_VULN(GiangVien *gv) {
    if (gv) {
        gv->is_deleted = 1;
        free(gv); // LOI CWE-416: free(gv) nhung khong cap nhat
            con tro o Module 2 thanh NULL!
    }
}
```

Listing 3: `delete_lecturer_VULN()` – CWE-416: `free()` không huỷ liên kết

```
void delete_lecturer_SAFE(GiangVien **gv_pptr) {
    if (gv_pptr && *gv_pptr) {
```

```
        (*gv_pptr)->is_deleted = 1;
        free(*gv_pptr);
        *gv_pptr = NULL; // Nullify pointer safely
    }
}
```

Listing 4: `delete_lecturer_SAFE()` – khắc phục: nhận `GiangVien **` để nullify con trỏ gốc

Listings 5 and 6 trích nguyên văn cặp hàm CWE-197 (dòng `tinh_tien_day.c:63` trong Bảng 5.2), đúng cặp hàm được chứng minh hình thức ở Check 4/5 (Mục 4.3).

```
char pack_class_size_byte_VULN(int soLuongSinhVien) {
    return (char)soLuongSinhVien;
}
```

Listing 5: `pack_class_size_byte_VULN()` – CWE-197: ép kiểu không kiểm tra phạm vi

```
int pack_class_size_byte_SAFE(int soLuongSinhVien, char *out_byte)
    {
    if (soLuongSinhVien < 0 || soLuongSinhVien > 127) {
        fprintf(stderr, "[SAFE ERROR] soLuongSinhVien=%d vuot pham
            vi char (0..127) - TU CHOI ep kieu de tranh Numeric
           Truncation!\n", soLuongSinhVien);
        return 0;
    }
    *out_byte = (char)soLuongSinhVien;
    return 1;
}
```

Listing 6: `pack_class_size_byte_SAFE()` – khắc phục: guard `[0,127]` trước khi ép kiểu

## 5.3 Kết quả Chương 3 – SMT

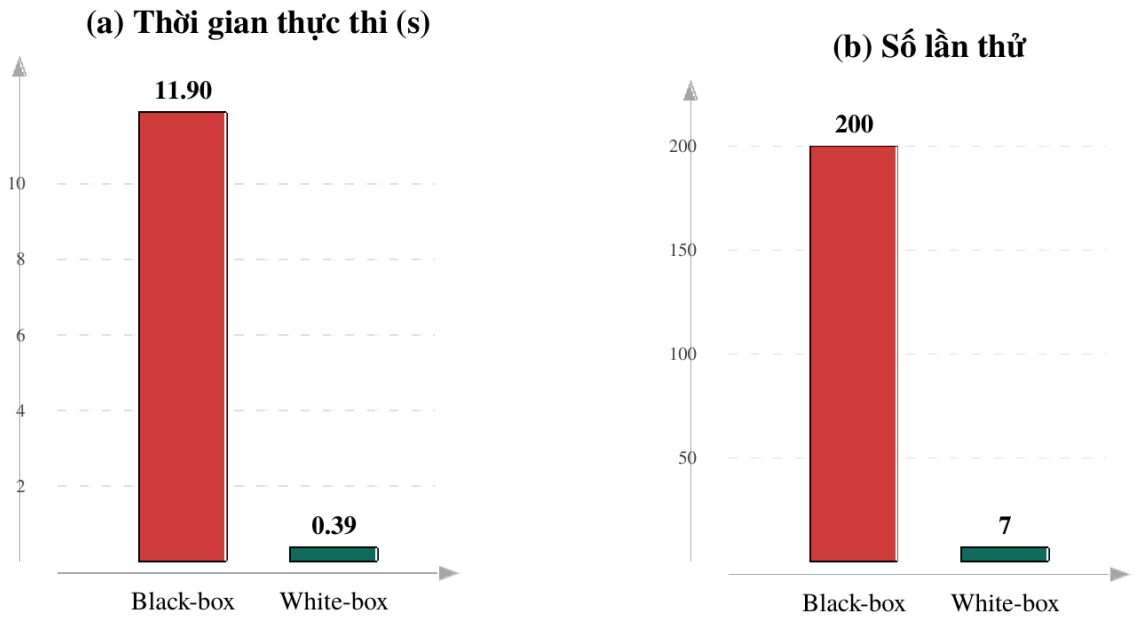
- **Check 1 SAT**: counterexample numClasses $= 20$, soTiet $= 120$, dinhMucTienChuan $= 127\,827$ VNĐ $\Rightarrow$ tổng thù lao vượt `INT32_MAX`.
- **Check 2 UNSAT**: chứng minh `long long` an toàn tuyệt đối trong toàn bộ ràng buộc doanh nghiệp (numClasses $\in [1, 100]$, dinhMucTienChuan tới 2 triệu VNĐ).

- **Check 3 UNSAT**: guard-check (`gv_ptr` NULL/đã xóa $\rightarrow$ lương $= 0$) hoạt động đúng trên chính công thức thật.
- **Check 4 SAT**: phản ví dụ soLuongSinhVien $= 128 \rightarrow$ byte $= -128$.
- **Check 5 UNSAT**: với guard $[0, 127]$ đúng trước, ép kiểu không bao giờ sai lệch.
- **Check 6 SAT**: $a = 0$ (lớp rỗng thật) và $b = 256$ (lớp có thật 256 sinh viên) đều cho byte $= 0$ – 2 tình huống nghiệp vụ hoàn toàn khác nhau không thể phân biệt sau khi xuất báo cáo.

## 5.4 Kết quả Chương 4 – Fuzzing

Bảng 5.3: So sánh Black-box vs. White-box (lần chạy đại diện, `logs/fuzzer_black/whitebox.log`)

| **Tiêu chí** | **Black-box** | **White-box (Z3)** |
|---|---|---|
| Số lần thử | 200 (ngẫu nhiên) | 7 (giải trực tiếp) |
| Số crash phát hiện (VULN) | 72/200 (36.0%) | 7/7 target (100%) |
| Số crash (SAFE) | 0/200 | – |
| Thời gian thực thi | 11.90 s | 0.3904 s |
| Tốc độ tăng tốc | mức cơ sở | $\approx 30.5\times$ nhanh hơn |




Hình 5.1: Black-box vs. White-box: (a) thời gian thực thi và (b) số lần thử cần để phát hiện đủ lỗ hổng đã biết (số liệu khớp Bảng 5.3)

Hai biểu đồ trên cùng minh họa một kết luận: White-box (thanh xanh, giải trực tiếp ràng buộc Z3) đạt **cùng độ chính xác 100% trên 7 mục tiêu** chỉ với 7 lần thử và 0.39s, trong khi Black-box (thanh đỏ, sinh input ngẫu nhiên) cần 200 lần thử và 11.90s – chênh lệch trực quan $\approx 30.5\times$ về thời gian mà không cần đọc số trong Bảng 5.3.

Riêng CWE-197 (không biểu hiện bằng crash mà bằng sai lệch dữ liệu âm thầm): black-box ngẫu nhiên cho **120/200 (60%)** sai lệch trên bản VULN, **0/200** trên bản SAFE (150/200 bị chặn an toàn). Bộ 17 giá trị biên (BVA) tất định khớp **17/17 (100%)** giữa byte thật và byte tính tay trên cả 2 bản, và tự động lộ ra **6 nhóm trùng lặp byte** trên bản VULN (ví dụ byte $= 0$ được tạo bởi cả 3 giá trị $-256, 0, 256$), **0 nhóm** trên bản SAFE.

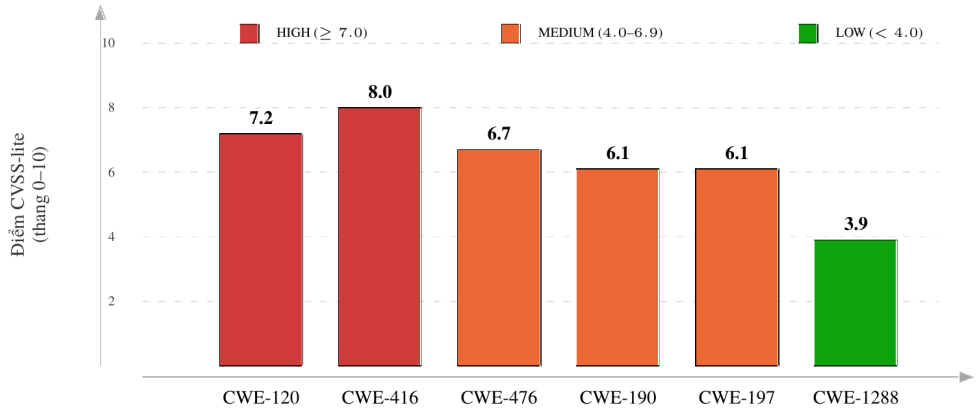
## 5.5 Bảng tổng hợp CWE và mức độ rủi ro (CVSS-lite)

Điểm CVSS-lite[3] dưới đây được tính theo công thức Score $= \min(\text{Exploitability} + \text{Impact}, 10)$ với Exploitability $= 8.22 \times AV \times AC \times PR \times UI$ và Impact $= 6.42 \times (1 - (1 - C)(1 - I)(1 - A))$ (cùng mô hình CVSS-lite đã dùng ở `lab3_secure_memory`), áp dụng $AV$=Local (chương trình CLI cục bộ, không phải dịch vụ mạng), $AC$=Low, $PR$=None, $UI$=None cho toàn bộ 6 CWE, và mức Impact (C/I/A) lấy theo đúng hậu quả *đã chứng minh được bằng thực nghiệm* ở các mục trên (không giả định RCE chưa

kiểm chứng).

Bảng 5.4: Tổng hợp CWE, vị trí trong đề tài và điểm CVSS-lite tự tính

| **CWE** | **Vị trí trong đề tài** | **CVSS-lite** | **Mức** |
|---|---|---|---|
| CWE-120[9] | Ch2 `strcpy`; Ch4 crash 72/200 | 7.2 | HIGH |
| CWE-416[13] | Ch2 `dangling_free`; Ch2.1 ASan `heap-use-after-free` thật | 8.0 | HIGH |
| CWE-476[14] | Ch2 `unchecked_nested_deref`; Ch4 Target 4 | 6.7 | MEDIUM |
| CWE-190[11] | Ch1 $S_5$; Ch3 Check 1 SAT / Check 2 UNSAT | 6.1 | MEDIUM |
| CWE-197[12] | Ch1 $P_5$; Ch3 Check 4/6; Ch4 BVA 17/17 | 6.1 | MEDIUM |
| CWE-1288[10] | Ch2 fallback `heSoBangCap=1.0f` | 3.9 | LOW |




Hình 5.2: So sánh điểm CVSS-lite giữa 6 CWE của đề tài (số liệu khớp Bảng 5.4)

Nhìn theo biểu đồ: CWE-416 (UAF) và CWE-120 (buffer overflow) là 2 lớp lỗi nghiêm trọng nhất (HIGH, $\geq 7.0$) – đúng với trực giác vì cả 2 đều có thể dẫn tới hỏng bộ nhớ ngay lập tức; CWE-197 và CWE-190 tuy nguy hiểm nhưng ở mức MEDIUM vì hậu quả quan sát được (sai lệch dữ liệu/DoS) chưa được chứng minh leo thang tới RCE trong phạm vi đề tài (xem hướng mở rộng ở Mục 6).

Bảng 5.5: Ví dụ thực tế minh họa cho mỗi lớp CWE

| **CWE** | **Ví dụ thực tế minh họa** |
|---|---|
| CWE-120 | CVE-2003-0352[7] – lỗi tràn bộ đệm trong Windows RPC DCOM, bị khai thác bởi worm Blaster |
| CWE-416 | CVE-2019-0708[8] (“BlueKeep”) – use-after-free trong Windows Remote Desktop Services |
| CWE-476 | Điển hình lớp lỗi DoS phổ biến; xem thêm các ví dụ tại trang CWE-476 của MITRE[14] |
| CWE-190 | Ariane 5 Flight 501 (1996)[5] – không phải mã CVE (xảy ra trước khi hệ thống CVE ra đời năm 1999) nhưng là sự cố tràn số nguyên kinh điển nhất từng được ghi nhận |
| CWE-197 | Chưa xác minh được một CVE công khai khớp chính xác; xem trang CWE-197 của MITRE[12] |
| CWE-1288 | Chưa xác minh được một CVE công khai khớp chính xác; xem trang CWE-1288 của MITRE[10] |

*Lưu ý phương pháp luận:* bảng trên cố ý **không** gán một mã CVE cụ thể cho CWE-476/197/1288 vì không xác minh được một CVE thực sự khớp chính xác với bối cảnh đề tài trong phạm vi kiểm chứng cho phép – thà để trống còn hơn trích dẫn một mã CVE không chắc chắn đúng.

## 5.6 Ma trận truy vết Test Case → Yêu cầu (IEEE 829)

Các mục 5.1–5.4 trên trình bày kết quả *theo từng chương/kỹ thuật*. Bảng dưới đây tổ chức lại đúng cùng số liệu đó theo hướng ngược lại – *theo từng yêu cầu (REQ/NFR)* – theo tinh thần đặc tả *Test Case Specification* của chuẩn IEEE 829-2008[4] (mã test case, điều kiện đầu vào, kết quả mong đợi, kết quả thực tế đối chiếu ngược lại nguồn log thật). Đây là góc nhìn truy vết hai chiều REQ ↔ Test Case còn thiếu ở Mục 2, nơi mới chỉ đặc tả yêu cầu mà chưa chỉ rõ từng yêu cầu được kiểm chứng bởi test case cụ thể nào.

Bảng 5.6: Ma trận truy vết Test Case → Yêu cầu (mọi kết quả thực tế trích từ `logs/*.log` thật)

| Mã TC | REQ | Input / Điều kiện | Kỳ vọng | Thực tế (nguồn log) | KL |
|---|---|---|---|---|---|
| TC-K1-01 | REQ-1 | BMC $P_1$, bản BUGGY (bước nhảy thẳng sang tính tiền, bỏ qua phân công) | UNSAT (không vi phạm) | **SAT** – phản ví dụ $S_0 \to S_5 \to S_4$ (`kripke_bmc.log`) | FAIL* |
| TC-K1-02 | REQ-1 | BMC $P_1$, bản FIXED | UNSAT | UNSAT (`kripke_bmc.log`) | PASS |
| TC-K1-03 | REQ-2 | BMC $P_2$, cả 2 bản | UNSAT | UNSAT cả 2 bản (`kripke_bmc.log`) | PASS |
| TC-K2-01 | REQ-3 | Scanner quét `strcpy` (`hoTen/maLop/bangCap/maGiangVien`) trên 3 file VULN | Phát hiện đủ cảnh báo CWE-120 | 5 cảnh báo CWE-120 đúng vị trí (`pattern_scanner.log`) | PASS |
| TC-K2-02 | REQ-3 | Scanner quét `main_fixed.c` | 0 cảnh báo | 0 cảnh báo – `[CLEAN]` (`pattern_scanner.log`) | PASS |
| TC-K4-01 | REQ-3 | White-box Target 1: `hoTen` dài 81 ký tự $>$ CON-1 (80) | Tìm điều kiện gây tràn | SAT ở lần thử 1 (`fuzzer_whitebox.log`) | FAIL* |
| TC-K4-02 | REQ-3 | White-box Target 2: `maLop` dài 21 ký tự $>$ CON-4 (20) | Tìm điều kiện gây tràn | SAT ở lần thử 2 (`fuzzer_whitebox.log`) | FAIL* |
| TC-K4-03 | REQ-3 | Black-box 200 lần ngẫu nhiên trên VULN | 0/200 crash | 72/200 crash, exit `0xC0000374` (`fuzzer_blackbox.log`) | FAIL* |
| TC-K4-04 | REQ-3 | Black-box 200 lần trên FIXED | 0/200 crash | 0/200 crash (`fuzzer_blackbox.log`) | PASS |
| TC-K2-03 | REQ-4 | Scanner quét `dangling_free` trên VULN | Phát hiện CWE-416 tại `giang_vien.c:32` | Phát hiện đúng vị trí (`pattern_scanner.log`) | PASS |
| TC-K4-05 | REQ-4 | White-box Target 3: `free(gv)` rồi vẫn đọc qua `gv_ptr` | Tìm điều kiện UAF | SAT ở lần thử 3 (`fuzzer_whitebox.log`) | FAIL* |

*xem tiếp trang sau*

*(tiếp theo Bảng 5.6)*

| Mã TC | REQ | Input / Điều kiện | Kỳ vọng | Thực tế (nguồn log) | KL |
|---|---|---|---|---|---|
| TC-K2-04 | REQ-4 | `memory_trace_demo.py`, đọc `gv->hoTen` sau `free()`, bản ASan-VULN | ASan phải bắt lỗi | `heap-use-after-free`, ABORTING, offset $+24$ byte $=$ `hoTen` (`memory_trace.log`) | FAIL* |
| TC-K2-05 | REQ-4 | `memory_trace_demo.py`, bản ASan-FIXED (`gv_ptr=NULL` trước free) | 0 vi phạm | 0 vi phạm (`memory_trace.log`) | PASS |
| TC-K2-06 | REQ-5 | Scanner quét `unchecked_nested_deref` trên VULN | Phát hiện CWE-476 tại `tinh_tien_day.c:13` | Phát hiện đúng vị trí (`pattern_scanner.log`) | PASS |
| TC-K4-06 | REQ-5 | White-box Target 4: `gv_ptr=NULL` rồi gọi `calculate_e2e_salary` | Tìm điều kiện NULL deref | SAT ở lần thử 4 (`fuzzer_whitebox.log`) | FAIL* |
| TC-K3-01 | REQ-6 | Check 1 (Z3): $\exists$ input hợp lệ làm tràn `int32` | SAT dự kiến (đang kiểm định code VULN) | SAT – numClasses=20, soTiet=120 (`smt_verify.log`) | FAIL* |
| TC-K3-02 | REQ-6 | Check 2 (Z3): cùng câu hỏi trên `long long` (bản vá) | UNSAT | UNSAT (`smt_verify.log`) | PASS |
| TC-K4-07 | REQ-6 | White-box Target 5: soTiet=16694179, dmt=2147583020 | SAT dự kiến | SAT ở lần thử 5 (`fuzzer_whitebox.log`) | FAIL* |
| TC-K1-04 | REQ-8 | BMC $P_5$, bản VULN | UNSAT | SAT – $S_3$ mang nhãn `byte_truncated` (`kripke_bmc.log`) | FAIL* |
| TC-K1-05 | REQ-8 | BMC $P_5$, bản FIXED | UNSAT | UNSAT (`kripke_bmc.log`) | PASS |
| TC-K3-03 | REQ-8 | Check 4 (Z3): $\exists n \in [0, 300]$ ép kiểu `char` sai giá trị | SAT dự kiến | SAT – $n = 128 \rightarrow$ byte $= -128$ (`smt_verify.log`) | FAIL* |
| TC-K3-04 | REQ-8 | Check 5 (Z3): cùng câu hỏi với guard $[0, 127]$ | UNSAT | UNSAT (`smt_verify.log`) | PASS |

*xem tiếp trang sau*

*(tiếp theo Bảng 5.6)*

| Mã TC | REQ | Input / Điều kiện | Kỳ vọng | Thực tế (nguồn log) | KL |
|---|---|---|---|---|---|
| TC-K3-05 | REQ-8, NFR-5 | Check 6 (Z3): $\exists\, a \neq b \in [0, 300]$ cùng cho 1 byte | UNSAT mong muốn (an toàn truy vết) | SAT – $a = 0, b = 256$ cùng byte $= 0$ (`smt_verify.log`) | FAIL** |
| TC-K4-08 | REQ-8 | Black-box CWE-197, 200 lần trên VULN | 0/200 sai lệch | 120/200 sai lệch, 60% (`fuzzer_blackbox.log`) | FAIL* |
| TC-K4-09 | REQ-8 | Black-box CWE-197, 200 lần trên FIXED | 0/200 sai lệch | 0/200 sai lệch, 150/200 chặn an toàn (`fuzzer_blackbox.log`) | PASS |
| TC-K4-10 | REQ-8, NFR-5 | BVA 17 giá trị biên cố định trên VULN | Byte thật khớp byte tính tay 17/17; không nên trùng lặp | 17/17 khớp lý thuyết, nhưng lộ 6 nhóm trùng lặp byte thật (`fuzzer_blackbox.log`) | PASS/FAIL** |
| TC-K4-11 | REQ-8, NFR-5 | BVA 17 giá trị biên trên FIXED | 0 nhóm trùng lặp | 0 nhóm trùng lặp (`fuzzer_blackbox.log`) | PASS |
| TC-K4-12 | REQ-9 | Black-box: `bangCap` ngẫu nhiên gồm giá trị không hợp lệ, trên VULN | Cần oracle xác nhận hệ thống xử lý đúng | Script chỉ đếm 75/200 lần input không hợp lệ được sinh ra, KHÔNG assert hành vi đúng/sai (`fuzzer_blackbox.log`) | QS† |
| TC-K2-07 | REQ-7 | (chưa có test case riêng) | Công thức tính lương đúng với mọi input hợp lệ | Check 1/2 chỉ chứng minh KHÔNG TRÀN SỐ, không chứng minh giá trị tính ra khớp công thức nghiệp vụ mong đợi | QS† |

*FAIL trên bản BUGGY/VULN là kết quả *đúng như thiết kế*: đây là bug injection có chủ đích để minh chứng công cụ phát hiện được lỗi thật, không phải lỗi của bản thân test case hay công cụ kiểm chứng. **FAIL phản ánh một giới hạn thiết kế thật (chỉ dùng 1 byte để lưu audit log, xem NFR-5 và Mục 6), không phải lỗi hiện thực có thể vá bằng guard đơn giản như CWE-197 gốc. †QS = Quan sát: đề tài hiện **chưa có oracle tự động** assert Pass/Fail cho REQ-7 (đúng công thức tính lương) và REQ-9 (validate `bangCap`) – đây là hai khoảng hở truy vết thật, được phát hiện chính nhờ việc lập ma trận này theo chiều REQ → Test Case, không phải hai yêu cầu này chưa từng được đề cập ở nơi khác trong báo cáo.

# 6 Thảo luận và Hạn chế

## 6.1 Giới hạn phương pháp

- **BMC $k = 5$ chưa đạt completeness threshold**: kết quả **UNSAT** của BMC chỉ chứng minh "không có vi phạm trong $k = 5$ bước", *không* loại trừ khả năng tồn tại vi phạm ở bước sâu hơn – đây chính là lý do $P_3$ (Liveness) ra SAT ở cả 2 bản (Mục 5): BMC hữu hạn không thể chứng minh một thuộc tính "cuối cùng phải xảy ra" một cách tuyệt đối, chỉ có thể chứng minh trong phạm vi $k$ bước đã duyệt.

- **Pattern scanner dùng regex/so khớp chuỗi, không phải phân tích AST/data-flow liên hàm (interprocedural) đầy đủ**: công cụ chỉ xét được guard-check trong phạm vi *một hàm*, nên sẽ bỏ sót lớp lỗi mà điều kiện bảo vệ nằm ở một hàm gọi *khác* với hàm chứa điểm truy cập nguy hiểm (ví dụ: một hàm kiểm tra `gv_ptr != NULL` rồi truyền tiếp con trỏ đã kiểm tra sang một hàm thứ hai đọc nó – scanner hiện tại sẽ không "nhớ" guard đó qua ranh giới lời gọi hàm). Đây là giới hạn cấu trúc cố hữu của một scanner một-pass; công cụ phù hợp để bổ trợ chứ không thay thế một static analyzer thật (Clang Static Analyzer, cppcheck với chế độ `-enable=all`).

- **Fuzzing trên Windows dùng exit code thay vì tín hiệu SIGSEGV kiểu POSIX**: Windows không có SIGSEGV theo nghĩa POSIX cho crash native; toàn bộ 72 mẫu crash CWE-120 quan sát được ở Chương 4 đều trả về **cùng một exit code 3221226356** (`0xC0000374` – `STATUS_HEAP_CORRUPTION`), không phải `0xC0000005` (`STATUS_ACCESS_VIOLATION`) như trực giác thường nghĩ về "buffer overflow gây segfault". Điều này phản ánh đúng cơ chế Windows CRT heap: ghi tràn qua `hoTen[80]` làm hỏng heap metadata liền kề, và lỗi chỉ bị phát hiện *muộn hơn* – tại lần gọi `free()` kế tiếp khi CRT heap manager kiểm tra tính toàn vẹn metadata – chứ không crash ngay tại dòng `strcpy`. Đây cũng là lý do một phần các lần thử "overflow" không crash ngay (75/200 rơi vào nhánh "logic error" thay vì crash): tuỳ lượng dữ liệu ghi tràn có chạm đúng vùng metadata bị kiểm tra hay không.

## 6.2 So sánh với lý thuyết đã học

- Kết quả $P_3$ SAT ở cả 2 bản đúng với lý thuyết *Soundness/Completeness* của BMC đã học ở Chương 1: BMC là *sound nhưng không complete* đối với Liveness ở độ sâu

hữu hạn – một kết quả SAT ở $P_3$ không tự động suy ra lỗi mã nguồn, phải phân biệt rõ với SAT của các thuộc tính Safety ($P_1, P_2, P_4, P_5$) vốn luôn tương ứng phản ví dụ thật.

- Tốc độ tăng tốc $\approx 30.5\times$ của white-box so với black-box phù hợp đúng dự đoán lý thuyết Chương 4: constraint solving trực tiếp (giải 7 ràng buộc đã biết) về bản chất là $O(1)$ theo số lần gọi solver, trong khi random fuzzing là $O(n)$ theo số lần thử cần để ngẫu nhiên trúng đúng điều kiện biên – tỉ lệ tăng tốc dao động khá lớn giữa các lần chạy thực tế (quan sát được từ $\approx 30\times$ đến $\approx 280\times$ tuỳ độ trễ khởi tạo tiến trình con của Windows ở từng lần đo) nhưng bản chất định tính không đổi.

## 6.3 Hướng cải thiện

- Áp dụng $k$**-Induction** (hoặc chuyển đổi sang Büchi automata) để chứng minh $P_3$ (Liveness) một cách tuyệt đối thay vì chỉ trong phạm vi $k = 5$ bước.
- Chạy lại bộ fuzzing trên **Linux** (qua WSL hoặc máy ảo) để thu được tín hiệu `SIGSEGV/SIGABRT` chuẩn POSIX, đối chiếu trực tiếp với hành vi `STATUS_HEAP_CORRUPTION` đã quan sát trên Windows – làm rõ liệu sự khác biệt về cơ chế phát hiện lỗi giữa 2 hệ điều hành có ảnh hưởng tới tỉ lệ crash đo được hay không.
- Thay thế/bổ trợ `pattern_scanner.py` bằng một static analyzer thật hỗ trợ phân tích interprocedural (cppcheck[1] `-enable=all` hoặc Clang Static Analyzer) để mở rộng phạm vi phát hiện ra ngoài giới hạn "một hàm" đã nêu ở mục 6.1.
- Mở rộng bảng CVSS-lite (Bảng 5.4) bằng việc thực sự khai thác (exploit) thử một lỗ hổng CWE-120 thành RCE có kiểm soát trong môi trường cô lập, để có cơ sở thực nghiệm nâng điểm Impact (C/I) thay vì chỉ dựa trên crash/DoS đã quan sát.

# 7 Kết luận

Đề tài đã phân tích và kiểm chứng an toàn mô-đun tính thù lao giảng viên của hệ thống DuongUniversity Management System bằng 4 kỹ thuật kiểm chứng bổ sung cho nhau – mô hình hóa hình thức (Kripke/BMC), phân tích tĩnh (pattern scanner), kiểm chứng hình thức bằng SMT (Z3), và kiểm thử động (fuzzing black-box/white-box) – áp dụng đồng nhất trên cùng một hệ thống E2E 3 module thay vì các ví dụ minh họa rời rạc.

Kết quả đạt được: phát hiện và khắc phục thành công **6 lớp lỗ hổng** (**CWE-120, CWE-416, CWE-476, CWE-190, CWE-1288, CWE-197**), xác nhận bằng **9/9 cảnh báo tĩnh** đúng vị trí trên code VULN và **0 cảnh báo** trên `main_fixed.c`; **5/5 thuộc tính BMC an toàn** trên bản FIXED (chỉ $P_3$ Liveness còn giới hạn kỹ thuật đã giải thích ở Mục 6); **6 phép chứng minh Z3** xác nhận hình thức cả tồn tại lỗi (Check 1, 4, 6 SAT) lẫn loại trừ hoàn toàn lỗi sau vá (Check 2, 3, 5 UNSAT); và bằng chứng thực nghiệm động qua fuzzing (72/200 crash trên bản lỗi, 0/200 trên bản vá) cùng chẩn đoán thật từ AddressSanitizer (`heap-use-after-free` được công cụ độc lập xác nhận, không phải tự báo cáo).

Điểm mấu chốt về phương pháp luận: **4 tầng kiểm chứng cùng đi đến một kết luận nhất quán** – Chương 1 chứng minh vi phạm ở mức logic điều khiển, Chương 2 chỉ đúng vị trí trên mã nguồn, Chương 3 chứng minh toán học trên toàn miền giá trị, Chương 4 xác nhận biểu hiện thật trên binary đã biên dịch. Không một kỹ thuật nào trong 4 kỹ thuật trên đủ để tự mình bao phủ toàn bộ không gian lỗi có thể có (mỗi kỹ thuật có giới hạn riêng, đã nêu ở Mục 6); sự kết hợp của cả 4 mới tạo thành một quy trình kiểm chứng an toàn phần mềm đáng tin cậy.

# Tài liệu tham khảo

[1] Cppcheck Team. Cppcheck: A tool for static c/c++ code analysis, 2024.

[2] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008)*, volume 4963 of *Lecture Notes in Computer Science*, pages 337–340. Springer, 2008.

[3] FIRST.Org, Inc. Common vulnerability scoring system version 3.1: Specification document. Technical report, Forum of Incident Response and Security Teams (FIRST), 2019.

[4] IEEE Computer Society. IEEE standard for software and system test documentation (IEEE std 829-2008). Technical report, Institute of Electrical and Electronics Engineers, 2008.

[5] Inquiry Board (ESA/CNES). Ariane 5 flight 501 failure – report by the inquiry board, 1996.

[6] LLVM Project. AddressSanitizer, 2024.

[7] MITRE Corporation. CVE-2003-0352: Buffer overflow in Microsoft RPC interface (“MSBlaster”), 2003.

[8] MITRE Corporation. CVE-2019-0708: Remote desktop services remote code execution vulnerability (“BlueKeep”), 2019.

[9] MITRE Corporation. CWE-120: Buffer copy without checking size of input (Classic Buffer Overflow), 2024. Accessed: 2026-09.

[10] MITRE Corporation. CWE-1288: Improper validation of consistency within input, 2024. Accessed: 2026-09.

[11] MITRE Corporation. CWE-190: Integer overflow or wraparound, 2024. Accessed: 2026-09.

[12] MITRE Corporation. CWE-197: Numeric truncation error, 2024. Accessed: 2026-09.

[13] MITRE Corporation. CWE-416: Use after free, 2024. Accessed: 2026-09.

[14] MITRE Corporation. CWE-476: NULL pointer dereference, 2024. Accessed: 2026-09.

[15] Vũ Quang Dũng. Softsec toolkit – giáo trình học phần CSE703093 an toàn phần mềm, 2026. Khoa Hệ thống thông tin, Trường Đại học Phenikaa.